Parametric EQ

User Manual & Technical Documentation

A 5-band parametric equalizer with a real-time pre-EQ spectrum analyzer, draggable EQ curve nodes, and a built-in test tone generator.

Version 0.1.0 · Built with JUCE 8.0.15 & C++17
July 2026

Contents

Part 1 — User Manual

This part describes Parametric EQ from the perspective of someone using it inside a DAW or as a Standalone application: what each control does, how the analyzer works, and how to get the lowest possible monitoring latency for live playing.

1. What This Plugin Does

Parametric EQ is a 5-band equalizer: five independently adjustable frequency bands that can boost or cut specific parts of the audible spectrum. Band 1 and Band 5 can each switch between a steep-cut Pass filter (removing everything below/above a frequency) and a gentle Shelf (boosting or cutting everything below/above a frequency). Bands 2, 3 and 4 are fixed Bell/Peak filters — the classic “boost or cut around one frequency” shape. A real-time spectrum analyzer shows the incoming signal alongside a live curve of exactly what the EQ is doing to it, and every band can be adjusted either with a rotary knob or by dragging its dot directly on that curve.

2. Interface Overview

The window is divided into five horizontal sections, top to bottom: the analyzer/curve panel, the test tone row, the latency readout, the five band panels, and the global controls row.

The Parametric EQ interface, annotated.

3. The Spectrum Analyzer & EQ Curve

The dark panel at the top overlays two things on a shared frequency (horizontal, 20 Hz–20 kHz, log-scaled) and gain (vertical, ±24 dB) grid:

- **Green trace** — a live FFT view of the incoming audio *before* any EQ is applied ("pre-EQ"). Useful for seeing exactly what frequency content is present in the source.
- **Orange curve** — the combined frequency response of all 5 bands plus the output trim: what the EQ is currently doing, end to end.
- **Orange dots** — one per band, sitting exactly on the orange curve at that band's current frequency and gain.

3.1 Dragging the curve nodes

Each of the 5 dots can be dragged directly with the mouse as an alternative to the rotary knobs:

- Dragging **horizontally** changes that band's frequency.
- Dragging **vertically** changes that band's gain.
- Hovering over a dot highlights it and changes the cursor to a hand; dragging enlarges it further.
- Dragging is clamped to that band's real range — e.g. Band 1's frequency cannot be dragged above 500 Hz, since that is its defined maximum.
- **Band 1 and Band 5 only:** while either is set to a Pass-filter type (High-Pass or Low-Pass), vertical dragging does nothing, because a pure Pass filter has no gain control — switch that band's Type to a Shelf to unlock vertical dragging for it.

Every drag is fully compatible with your DAW's automation — it behaves exactly like moving the corresponding rotary knob, including undo history and automation recording.

4. The Five Bands

Each band panel shows the band name, its Frequency/Gain/Q rotary knobs with a live numeric readout underneath each, and — for Band 1 and Band 5 only — a Type and Slope dropdown above the knobs.

Band 1 — Low

Switchable between **High-Pass** (removes everything below the set frequency) and **Low Shelf** (boosts/cuts everything below the set frequency by the Gain amount). The Slope control (12/24/36 dB/octave) sets how steep the cut or shelf transition is, and applies in both modes.

Parameter	Range	Default	Notes
Type	High-Pass / Low Shelf	High-Pass	Gain is inactive while in High-Pass mode
Frequency	20 – 500 Hz	80 Hz	
Gain	–24 to +24 dB	0 dB	Only audible in Low Shelf mode
Q	0.1 – 10	0.707	Resonance/sharpness of the transition
Slope	12 / 24 / 36 dB/oct	12 dB/oct	Steepness of the cut or shelf

Bands 2–4 — Low-Mid, Mid, High-Mid

Fixed Bell/Peak filters: boost or cut centred on the set frequency, with Q controlling how wide or narrow that boost/cut is (higher Q = narrower).

Parameter	Range	Default	Notes

Frequency	20 Hz – 20 kHz	300 / 1000 / 3000 Hz	One row per band (2 / 3 / 4)
Gain	–24 to +24 dB	0 dB	
Q	0.1 – 10	1.0	

Band 5 — High

The mirror image of Band 1: switchable between **Low-Pass** (removes everything above the set frequency) and **High Shelf** (boosts/cuts everything above it), with the same slope options.

Parameter	Range	Default	Notes
Type	Low-Pass / High Shelf	Low-Pass	Gain is inactive while in Low-Pass mode
Frequency	500 Hz – 20 kHz	8000 Hz	
Gain	–24 to +24 dB	0 dB	Only audible in High Shelf mode
Q	0.1 – 10	0.707	
Slope	12 / 24 / 36 dB/oct	12 dB/oct	

5. Built-in Test Tone

The row below the analyzer lets you replace the incoming audio with a clean sine wave, useful for testing the EQ's effect on a known signal without needing an external source. Turning it on silences whatever audio was coming in and substitutes the tone instead — it does not mix with the input.

Parameter	Range	Default	Notes
Test Tone (on/off)	—	Off	Replaces the input signal entirely when on
Freq	20 Hz – 20 kHz	1000 Hz	
Level	–60 to 0 dB	–18 dB	Kept conservative by default to protect your ears/speakers

The test tone is injected before the spectrum analyzer's tap point, so it shows up on the green trace too, and it keeps playing even if Bypass is engaged.

6. Bypass & Output Trim

The bottom row has a **Bypass** toggle (mutes all EQ processing for an instant A/B comparison against the dry signal — the analyzer keeps running throughout) and an **Output Trim** slider (–24 to +24 dB) that adjusts the overall level after all 5 bands, useful for compensating for gain added or removed by the EQ.

7. Latency & Getting the Lowest Round-Trip Time

The plugin's own processing adds effectively zero latency — every filter is a direct-form IIR biquad with no lookahead or internal buffering. What you actually feel when playing live through the EQ is your audio interface's own round-trip time, shown live in the readout just below the test tone row (format: *buffer size @ sample rate, driver name*).

7.1 If your interface has an ASIO driver (Windows)

Open **Options** → **Audio/MIDI Settings** in the Standalone window and switch the audio device type to **ASIO**, then select your interface's own ASIO driver (e.g. “Focusrite USB ASIO”) rather than Windows Audio/DirectSound. ASIO talks to the interface directly instead of going through the shared Windows audio mixer, which is what allows very small buffer sizes (64–128 samples) without crackling. Lower the buffer size until the readout shows a round-trip time you're comfortable playing with — under ~20 ms is generally considered playable for most instruments.

Without a real ASIO driver, Windows Audio/DirectSound have a much higher latency floor and typically distort or drop out if pushed to very small buffer sizes.

8. Saving Your Settings

All parameter values (every band, bypass, output trim, and the test tone settings) are saved and restored automatically as part of your DAW project or Standalone session state — there is no separate preset browser in this version.

9. Supported Formats & Platforms

- **Standalone application** — runs on its own, for practicing/testing without a DAW.
- **AAX** (Pro Tools) — requires Avid's AAX SDK to build; not needed for Standalone use.
- Windows (primary) and macOS.

Part 2 — Technical Documentation

This part documents how Parametric EQ is implemented: the architecture, the DSP, the analyzer pipeline, the UI layer, the build system, and the automated test coverage. It assumes familiarity with C++ and the JUCE framework.

1. Architecture Overview

The plugin follows JUCE's standard AudioProcessor / AudioProcessorEditor split, with a hard rule that DSP and UI never touch each other directly — they communicate exclusively through a **juce::AudioProcessorValueTreeState** (APVTS), which is JUCE's parameter management and automation/undo layer.

```
PluginProcessor (AudioProcessor)
|-- apvts (AudioProcessorValueTreeState) <- single source of truth for all params
|-- LowBand, PeakBand x3, HighBand <- one DSP class per band
|-- juce::dsp::Gain<float> <- output trim
|-- TestToneGenerator
\-- SpectrumAnalyzerFifo <- lock-free pre-EQ tap for the UI

PluginEditor (AudioProcessorEditor)
|-- AnalyzerPanelComponent
| |-- SpectrumAnalyzerComponent <- raw FFT trace (reads the Fifo)
| \-- EqCurveOverlayComponent <- axes, composite curve, draggable dots
|-- TestToneControlsComponent, LatencyDisplayComponent
|-- BandControlComponent x5
\-- GlobalControlsComponent
```

Every DSP class exposes a pure, **static** magnitude-query method (e.g. `PeakBand::getMagnitudeForFrequency`) alongside its real-time `process()` path. These static methods rebuild filter coefficients independently from APVTS parameter values and are what the UI calls to draw the EQ curve — the UI thread never reads a DSP object's live, audio-thread-owned state. This was a deliberate fix for a real bug found during development (see §7).

2. Build System

- **CMake** (3.22+) with **CPM.cmake** (vendored in `cmake/CPM.cmake`) fetching **JUCE 8.0.15**, pinned deliberately over the newer 9.0.0 for ecosystem/tutorial maturity.
- **C++17** throughout.
- **Standalone** format builds by default with zero extra setup. **AAX** is gated behind a `PARAMETRIC_EQ_BUILD_AAX` CMake option plus an `AAX_SDK_PATH` cache variable; enabling it without a valid path fails fast with `FATAL_ERROR` rather than attempting any network fetch, since the AAX SDK is proprietary and requires an Avid developer agreement.
- `JUCE_ASIO=1` is explicitly enabled (off by default in JUCE) so real audio interfaces expose their native low-latency ASIO driver on Windows.
- The app/taskbar icon is wired via `ICON_BIG`/`ICON_SMALL` pointing at `Resources/AppIcon.png`; JUCE converts it to a Windows `.ico` resource at build time.

- A separate `ParametricEQ_Tests` executable links only the DSP/parameter sources (no UI, no plugin-client wrapper) and runs via `juce::UnitTestRunner`.

Known project-specific build quirk: incremental CMake/MSBuild rebuilds of the Standalone target have, on this project, occasionally produced a stale/corrupted executable with visibly wrong UI rendering (text colour) despite unchanged, correct source — reproduced and fixed multiple times by deleting the entire `build/` directory and reconfiguring from scratch. The root cause was not fully isolated; treat a full clean rebuild as the standard troubleshooting step before suspecting a source-level bug.

3. Parameter Reference (APVTS)

All 24 parameters are created in `Source/Parameters/ParameterLayout.cpp` and identified by flat string IDs in `ParameterIDs.h` (kept AAX-safe: no special characters). Frequency parameters use a log-skewed `NormalisableRange` centred at their default value; Q is skewed toward 1.0.

Parameter	Range	Default	Notes
band1Type	choice: High-Pass/Low Shelf	High-Pass	
band1FreqHz / GainDb / Q / Slope	20-500 Hz / ±24dB / 0.1-10 / 12-36	80 / 0 / 0.707 / 12	
band2..4 FreqHz / GainDb / Q	20-20000 Hz / ±24dB / 0.1-10	300,1000,3000 / 0 / 1.0	No Type/Slope
band5Type	choice: Low-Pass/High Shelf	Low-Pass	
band5FreqHz / GainDb / Q / Slope	500-20000 Hz / ±24dB / 0.1-10 / 12-36	8000 / 0 / 0.707 / 12	
outputGainDb	±24 dB	0	
bypass	bool	false	
testToneEnabled / FreqHz / GainDb	bool / 20-20000 Hz / -60-0 dB	false / 1000 / -18	

4. DSP Layer

4.1 SlopeFilterChain

Shared engine behind Band 1 and Band 5, cascading up to 3 `juce::dsp::IIR::Filter<float>` stages to give a selectable 12/24/36 dB/octave response:

- **Pass modes** (High-Pass/Low-Pass) use `juce::dsp::FilterDesign<float>::designIIR...HighOrderButterworthMethod` with order = slope/6, giving a true per-stage Butterworth response rather than naive identical-biquad repetition.
- **Shelf modes** with slope > 12 cascade N identical-frequency `IIR::Coefficients::makeLowShelf/makeHighShelf` stages, splitting the total gain evenly across stages (`gainDb / numStages`) so the plateau gain stays correct while the transition steepens with more stages — JUCE has no built-in higher-order shelf designer, so this is a pragmatic, documented interpretation.
- Coefficient construction is factored into `static buildStageCoefficients()`/`getMagnitudeForFrequency()` methods, called by both the real-time `update()` path and the UI's magnitude queries — one source of truth, no duplicated math.

4.2 LowBand / HighBand / PeakBand

`LowBand` and `HighBand` each own one `SlopeFilterChain` and translate their Type parameter into a `SlopeFilterKind`. `PeakBand` (used three times for bands 2–4) is a single biquad via `IIR::Coefficients::makePeakFilter`. All three share the same pattern: each continuous parameter (freq/gain/Q) is driven through a `juce::SmoothedValue<float>` (20ms ramp), and filter coefficients are only rebuilt when the ramped, block-effective value actually changes from a cached last value — zero recomputation at rest, avoiding unnecessary work in `processBlock()` while still following the project's no-allocation-at-rest rule.

4.3 TestToneGenerator

Wraps a `juce::dsp::Oscillator<float>` (raw sine, no lookup table). A real bug was caught and fixed here during development: `Oscillator::process()` *adds* to the existing buffer content on a `ProcessContextReplacing` rather than overwriting it (it is designed as an FM-style modulator). `TestToneGenerator::process()` therefore calls `buffer.clear()` before running the oscillator, so the tone genuinely replaces the input rather than mixing with it. Level is applied via `AudioBuffer::applyGainRamp` driven by a `SmoothedValue`, for click-free level changes.

5. Analyzer Pipeline

5.1 SpectrumAnalyzerFifo

Lock-free, allocation-free hand-off of the pre-EQ signal from the audio thread to the UI thread. The audio thread mixes the block to mono into a pre-allocated scratch buffer and writes it into a `juce::AbstractFifo`-backed ring buffer; the UI thread drains it, accumulates a 2048-sample Hann-windowed block, and hands it back. If the UI stalls and the ring fills up, excess samples are simply dropped (a display-lag cost, not a correctness issue).

5.2 SpectrumAnalyzerComponent

Runs a 30Hz `juce::Timer` that pulls a block from the Fifo, runs a **zero-padded** FFT, and maps bins to a 512-point display array. The real accumulation window stays at 2048 samples (no added latency), but the transform itself is zero-padded 8x to 16384 points (order 14) before `performFrequencyOnlyForwardTransform`, sinc-interpolating far more display points between the real bins. This fixed a visibly “blocky” staircase artifact that was most visible at low frequencies, where 2048 real samples at 48kHz gives only ~23Hz/bin (the 20-50Hz range spans barely one bin unpadded). dB normalisation is deliberately computed against the real (unpadded) sample count, since zero-padding interpolates extra points but does not change the DFT's underlying magnitude scaling.

5.3 AnalyzerMapping

Header-only shared frequency/gain ↔ pixel mapping (log-scale frequency, linear dB), included by the trace, the axis gridlines, and the composite curve so all three can never visually disagree about where a given Hz or dB value sits on screen. Provides `frequencyToX/xToFrequency` and `dbToY/yToDb` (the latter added specifically to support dragging the curve nodes).

5.4 EqCurveOverlayComponent

Draws the frequency/dB axis gridlines, the composite curve (built by sampling `PluginProcessor::getMagnitudeResponseDb()` at ~300 points across the panel width, once per repaint, then mapped through AnalyzerMapping), and the 5 draggable band marker dots. Mouse interaction:

- `mouseDown` hit-tests all 5 marker positions (shared `bandMarkerPosition()` helper, also used for drawing) against a 12px radius and begins a parameter “change gesture” on the hit band's frequency (and gain, if applicable) parameter.
- `mouseDrag` converts the new mouse position back to Hz/dB via AnalyzerMapping, clamps it against the parameter's own live `NormalisableRange` (read via `RangedAudioParameter::getNormalisableRange()` — never re-hardcoded per band), and writes it with `setValueNotifyingHost(convertTo0to1(value))`.
- Gain dragging is a no-op for Band 1/5 while that band's Type parameter reads as a Pass filter (checked live, at drag time, against the raw APVTS choice index) — the DSP never reads gain in that mode, so allowing the drag would be misleading.
- `mouseUp` ends the gesture. This gesture-based write path (identical to what `SliderAttachment` uses internally) is not optional: a raw `getRawParameterValue()->store(value)` would silently skip both host automation recording and the `AudioProcessorParameter::Listener` notification that keeps the rotary knobs visually in sync during a drag.

5.5 AnalyzerPanelComponent

A thin z-order container: `SpectrumAnalyzerComponent` (the raw trace) underneath, `EqCurveOverlayComponent` (axes/curve/dots, transparent background) on top, both at identical bounds. Kept as two components rather than merged, since they have genuinely different data sources: an audio-thread-fed FFT ring buffer versus UI-thread parameter reads.

6. UI Layer

- **Theme.h** — single shared colour palette (background gradient, glass-panel fill/border/sheen/shadow, accent orange, text colours), mirroring the AnalyzerMapping.h pattern of one file for one cross-cutting concern.
- **GlassPanel.h** — free function drawing the translucent rounded “glass card” background (drop shadow, fill, clipped sheen gradient, border) used by every control panel's `paint()`.
- **ModernLookAndFeel** — a `juce::LookAndFeel_V4` subclass applied once at the editor and inherited by every child. Overrides `drawRotarySlider` (dim background arc + orange value arc + line indicator), `drawLinearSlider` (pill track), and `drawToggleButton` (pill switch); ComboBox/PopupMenu are left to the stock painter, themed purely via colour IDs set in the constructor.
- **BandControlComponent, GlobalControlsComponent, TestToneControlsComponent, LatencyDisplayComponent** — each a small, single-purpose `juce::Component` wiring stock JUCE widgets to APVTS via `SliderAttachment`/`ComboBoxAttachment`/`ButtonAttachment`, plus a one-line `GlassPanel::draw()` call in their own `paint()`.
- **LatencyDisplayComponent** specifically reads the live device via `juce::StandalonePluginHolder::getInstance()` (a fully header-only, inline JUCE class — safe to reference even in non-Standalone builds, where it simply returns `nullptr` and the readout shows “host-controlled” instead).

7. Thread-Safety Notes

A real use-after-free race was identified and designed around during development: reading a DSP band's live `Filter::coefficients` (a `ReferenceCountedObjectPtr`) from the UI thread while the audio thread concurrently reassigns it is unsafe — the atomic refcount only protects the object's reference count, not the raw pointer swap itself, and JUCE's own header comments confirm the caller must synchronise this manually. The fix, applied throughout the DSP layer, is that the UI never touches live audio-thread objects at all: every DSP class instead exposes pure, `static` query methods that independently rebuild fresh, UI-thread-local coefficients from APVTS parameter values, using the exact same coefficient-building logic as the real-time path (extracted into shared static builder functions specifically to avoid duplicating that math). No locks are used anywhere in this plugin.

8. Automated Tests

`Tests/DspUnitTests.cpp` builds a separate, UI-free executable (`ParametricEQ_Tests`) using `juce::UnitTestRunner`:

- Parameter layout: correct total parameter count, key IDs resolve.
- SlopeFilterChain: output stays finite across all three slope options for both High-Pass and Low-Shelf; magnitude at a Butterworth cutoff reads back close to the theoretical –3dB point.
- PeakBand/LowBand: output stays finite while gain is automated or type is switched across many blocks (smoothing/dirty-check correctness).
- PeakBand magnitude query reads back its own configured gain at its centre frequency.
- TestToneGenerator: disabled generator leaves the buffer untouched (regression test for the buffer.clear() fix in §4.3); enabled generator produces a finite, level-bounded sine.

9. Known Limitations / Out of Scope for V1

- No preset system or preset browser.
- No per-band solo/mute.
- No undo/redo beyond what the host DAW itself provides via automation gestures.
- No oversampling.
- VST3/AU are not built in V1 (Standalone + AAX only); JUCE's plugin client would make adding them straightforward if needed later.

10. Source File Manifest

File	Responsibility
`PluginProcessor.h/.cpp`	AudioProcessor: composes all DSP, APVTS, processBlock, state save/restore, magnitude-query API
`PluginEditor.h/.cpp`	AudioProcessorEditor: owns and lays out all UI components, LookAndFeel lifecycle
`Parameters/ParameterIDs.h`	Flat string parameter ID constants
`Parameters/ParameterLayout.cpp`	Builds the 24-parameter APVTS ParameterLayout

`DSP/SlopeFilterChain.h/.cpp`	Cascaded biquad engine for bands 1 & 5's 12/24/36 dB/oct Pass/Shelf response
`DSP/LowBand.h/.cpp`	Band 1 wrapper: High-Pass/Low Shelf + smoothing
`DSP/HighBand.h/.cpp`	Band 5 wrapper: Low-Pass/High Shelf + smoothing
`DSP/PeakBand.h/.cpp`	Bands 2-4: fixed Bell/Peak filter + smoothing
`DSP/TestToneGenerator.h/.cpp`	Sine test tone that replaces the input signal when enabled
`Analyzer/SpectrumAnalyzerFifo.h/.cpp`	Lock-free audio-thread to UI-thread pre-EQ signal hand-off
`Analyzer/SpectrumAnalyzerComponent.h/.cpp`	Zero-padded FFT + raw spectrum trace rendering
`Analyzer/AnalyzerMapping.h`	Shared frequency/dB <-> pixel mapping
`Analyzer/EqCurveOverlayComponent.h/.cpp`	Axis grid, composite curve, draggable band marker dots
`Analyzer/AnalyzerPanelComponent.h/.cpp`	Z-order container: trace + overlay
`UI/Theme.h`	Shared colour palette
`UI/GlassPanel.h`	Shared translucent panel background renderer
`UI/ModernLookAndFeel.h/.cpp`	Custom rotary/linear slider and toggle button painters
`UI/BandControlComponent.h/.cpp`	One band's Freq/Gain/Q knobs + Type/Slope combo boxes
`UI/GlobalControlsComponent.h/.cpp`	Bypass toggle + Output Trim slider
`UI/TestToneControlsComponent.h/.cpp`	Test tone enable/Freq/Level controls
`UI/LatencyDisplayComponent.h/.cpp`	Live round-trip latency readout
`Tests/DspUnitTests.cpp`	juce::UnitTestRunner-based DSP/parameter test suite
`CMakeLists.txt`	Build configuration: CPM/JUCE fetch, targets, AAX gating, icon, tests

---

Generated to document the state of the project as of July 2026.